Project Report on

SMART AI PHISHING URL BLOCKER

Machine Learning Based Phishing URL Detection System

MAJOR PROJECT REPORT

Submitted in partial fulfillment of the requirements for the degree of



GUJARAT UNIVERSITY

Bachelor of Computer Applications (BCA)

Developed By
Shaikh Aksha (202312101997)

(Sem – VII Bachelor of Computer Application)

Asia pacific BCA College

A Project Report On

SMART AI PHISHING URL BLOCKER

Submitted to Faculty of
Bachelor of Computer Application

In partial fulfillment of the requirement of the award for the degree Of

Asia Pacific®
Institute of Management
Ahmedabad

Bachelor Of Computer Applications

Gujarat University

Under the guidance of

Name of Faculty:

Submitted by:

Name	Enrollment no.
Shaikh Aksha	**202312101997**

CERTIFICATE

*"This is to Certify that the following students of Asia Pacific BCA College of Computer Applications, from BCA SEMESTER VII (Honors BCA) have satisfactory completed Machine Learning Project On "**SMART AI PHISHING URL BLOCKER**".*

Vice – Principal In charge

Name and signature
Guide Principal I/C

Date: _____

Place:_____

2. ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my college and the faculty members who provided guidance and support during the development of this Machine Learning project, “Smart AI Phishing Website & Malicious URL Blocker.” Their academic instructions helped me understand the stages of machine learning system design, implementation, evaluation, and documentation.

I am thankful to my project guide for the suggestions and encouragement provided throughout the project work. I also acknowledge the learning resources, Python libraries, and open-source tools used to develop and test the project.

This project gave me practical experience in preparing a dataset, removing duplicate records, exploring data, training supervised machine learning models, comparing evaluation metrics, applying K-Means clustering, and connecting a trained model to a Flask web application.

Finally, I would like to thank everyone who supported me directly or indirectly in completing this project. The results and descriptions in this report are based on the notebook, application files, and README included with the project.

Student Name : Shaikh Akshabanu Zahirabbas.
Class / Semester: BCA Sem 7
Roll Number : 4010
Date : 15-10-2026

3. INDEX

Sr. No.	Contents	Page No.
1	College certificate	3
2	Acknowledgement	4
3	Index Page	5
4	Introduction	6
5	Dataset selection	7
6	Data Preprocessing	8
7	Exploratory Data Analysis (EDA)	9
8	Visualization using Matplotlib and Seaborn	10
9	Model Evaluation and Performance metrics	11
10	Model Building 10.1 Logistic Regression 10.2 Random Forest 10.3 Support Vector Machine (SVM) 10.4 Model training and testing 10.5 Hyperparameter tuning 10.6 Performance comparison 10.7 Best model selection	12 13
11	Advanced techniques and Pattern discovery	14
12	Conclusion & future work	15
13	Application Screenshots	16 , 17, 18
14	Bibliography	19

4. INTRODUCTION

Phishing is a form of online deception in which a malicious website or URL attempts to appear trustworthy and may be used to obtain sensitive information. A URL can contain clues such as unusual length, IP-address usage, suspicious symbols, domain characteristics, redirects, or webpage properties. Checking these clues manually can be difficult for ordinary users.

The project "Smart AI Phishing Website & Malicious URL Blocker" applies machine learning to classify URL-related records as phishing or legitimate. It uses a dataset containing encoded characteristics of websites and trains supervised classifiers to learn patterns associated with the two classes. The selected model is then connected to a Flask web application, where a user can submit a URL and receive a classification with a confidence value.

4.1 Problem definition

The main problem is to identify whether a website/URL is likely to be phishing or legitimate by analysing its features. A simple rule-based approach may miss combinations of signals. This project therefore uses a trained classifier that considers multiple features together. The output is a prediction to support user awareness; it should not be treated as a guarantee that a website is safe or malicious.

4.2 Objective of project

Prepare and validate a phishing website dataset for machine learning.
Explore the data using summary statistics, class distribution, plots, and correlation analysis.
Train and compare supervised classification models using a consistent train-test split.
Evaluate models with Accuracy, Precision, Recall, F1-score, ROC-AUC, and a confusion matrix.
Apply K-Means clustering to explore patterns in the feature data without using the target during clustering.
Integrate the selected tuned Random Forest model into a Flask application for URL prediction.
The practical work follows a complete pipeline: dataset preparation, exploratory analysis, model training, model comparison, clustering, model saving, and web-app integration.

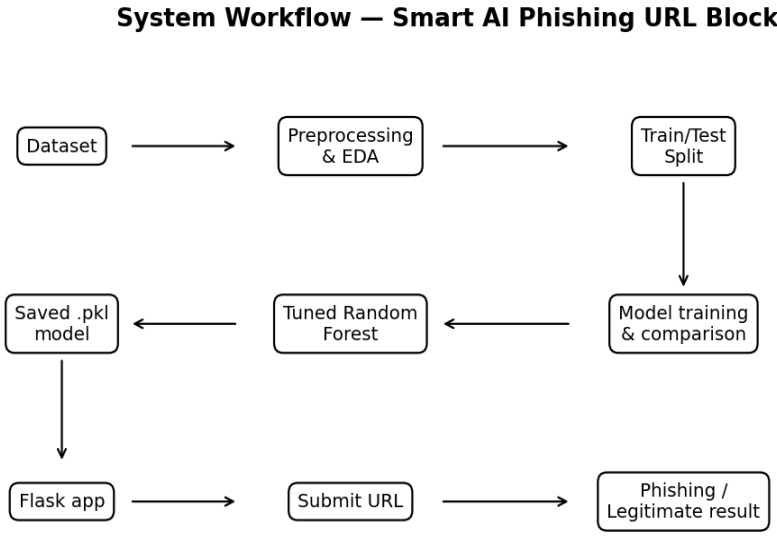


Figure 1. Project workflow based on the notebook and Flask appl

5. DATASET SELECTION

The project notebook loads a file named “Phishing_Websites_Data.csv”. It contains numerical, encoded website/URL characteristics and a target column named “Result”. The notebook records 11,055 rows in the original dataset. After duplicate removal, 5,849 records remain for analysis and model development.

5.1 Dataset structure

Item	Details recorded in project
Dataset file	Phishing_Websites_Data.csv
Original records	11,055
Records after duplicate removal	5,849
Input features	30
Target variable	Result
Total columns after cleaning	31 (30 features + target)
Missing values after cleaning	0
Duplicate rows after cleaning	0
Phishing class	Result = -1
Legitimate class	Result = 1

5.2 Feature description

The input columns represent characteristics of URLs and websites. Examples include having_IP_Address, URL_Length, Shortining_Service, having_At_Symbol, double_slash_redirecting, Prefix_Suffix, having_Sub_Domain, SSLfinal_State, Domain_registeration_length, Request_URL, URL_of_Anchor, Links_in_tags, SFH, Redirect, age_of_domain, DNSRecord, web_traffic, Google_Index, and other domain/page-related indicators.

Most features are already represented by encoded numerical values such as -1, 0, and 1. The target “Result” gives the known class used for supervised training. The notebook identifies 3,019 phishing records (51.62%) and 2,830 legitimate records (48.38%) after cleaning. The dataset file is referenced by the notebook. The report does not claim a particular download URL or publisher because that exact source is not identified in the project files reviewed.

6. DATA PREPROCESSING

Data preprocessing was performed before model training so that the analysis would use a consistent and validated dataset. The notebook loads the CSV with pandas and checks its structure, column names, data types, missing values, duplicate rows, and target distribution.

6.1 Cleaning steps

Load the dataset from Phishing_Websites_Data.csv using pandas.
Inspect the dataset shape, column names, and information about the columns.
Check missing values across the dataset.
Identify duplicate rows and remove them from the working dataset.
Confirm the final record count, feature count, missing-value count, and duplicate count.
Separate the input features (X) from the target variable (y = Result).

6.2 Result after cleaning

Validation item	Notebook result
Original dataset size	11,055 records
Duplicate rows identified	5,206
Cleaned dataset size	5,849 records
Input features	30
Target column	Result
Missing values after cleaning	0
Duplicate rows after cleaning	0

The cleaned dataset has no missing values and no remaining duplicate rows. Its numerical feature representation makes it suitable for the classification algorithms used in the notebook. The target classes are relatively balanced, so both phishing and legitimate examples are represented in the training and test data.

6.3 Train-test split

For supervised model evaluation, the notebook uses train_test_split with test_size=0.2, random_state=42, and stratify=y. This results in 4,679 training records and 1,170 testing records. Stratification preserves the target-class proportions across the two subsets. The test subset is held out for evaluating model predictions.

7. EXPLORATORY DATA ANALYSIS (EDA)

Exploratory Data Analysis was used to understand the cleaned data before selecting and evaluating models. The notebook combines numerical summaries, class counts, feature means, and correlation analysis.

7.1 Univariate Analysis

Univariate analysis examines one variable at a time. The notebook checks the distribution of the target variable and calculates counts and percentages for the phishing and legitimate classes. The cleaned data contains 3,019 phishing records (51.62%) and 2,830 legitimate records (48.38%). This indicates a relatively balanced target distribution, without a major class imbalance.

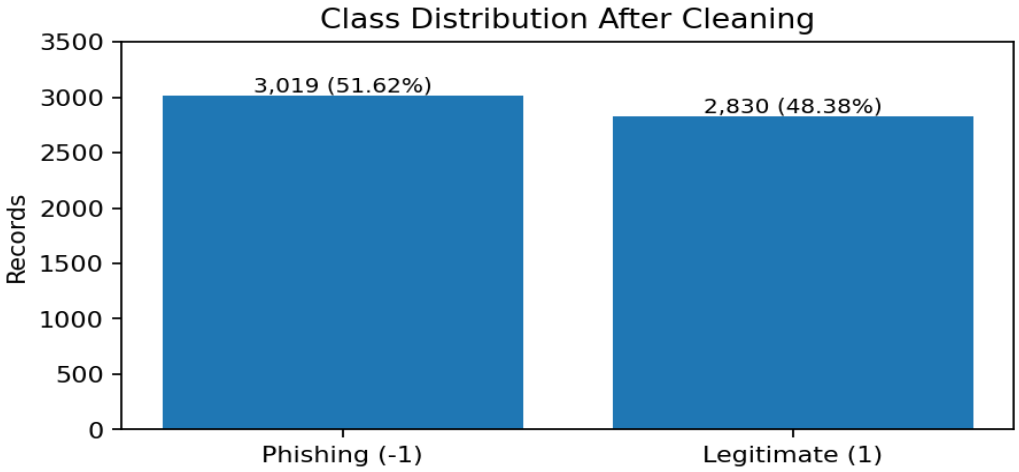


Figure 2. Target-class distribution after duplicate removal.

7.2 Bivariate Analysis

Bivariate analysis considers relationships between two variables. The notebook examines how individual input features relate to the target variable and visualizes feature means. This helps identify features whose encoded values differ between phishing and legitimate records. Such relationships are exploratory and do not, by themselves, prove that a feature causes a URL to be malicious.

7.3 Statistical Summary

The descriptive summary is produced using pandas statistical functions. After cleaning, the data has 5,849 rows and 31 columns, with 5,849 non-null observations for every column. The features are mostly encoded numerical values. The summary helps review ranges, central values, and variation in the available columns.

7.4 Correlation Analysis

The notebook calculates correlations between input features and the target and displays a correlation heatmap. SSLfinal_State and URL_of_Anchor show relatively strong positive correlations with the target in the notebook's analysis. Prefix_Suffix, having_Sub_Domain, web_traffic, SFH, and Links_in_tags also show noticeable relationships. Domain_registeration_length has a negative relationship with the target in the recorded analysis.

Because the target uses -1 for phishing and 1 for legitimate, the sign of a correlation must be interpreted with this encoding in mind. Correlation indicates association, not proof of causation.

8. VISUALIZATION USING MATPLOTLIB AND SEABORN

Matplotlib and Seaborn are used in the notebook to present dataset distributions and model results visually. Visualizations make it easier to understand class balance, feature relationships, and model performance than inspecting numeric tables alone.

8.1 Class distribution

A count plot/bar-style visualization shows the number of phishing and legitimate records in the cleaned dataset. The plotted distribution reflects 3,019 phishing records and 2,830 legitimate records. The two classes are relatively balanced.

8.2 Feature analysis

The notebook calculates and plots feature means to compare encoded characteristics. The purpose is to explore which website indicators may differ between the target classes. The chart is an exploratory aid; a feature mean alone is not sufficient to decide whether an individual URL is safe.

8.3 Correlation heatmap

A Seaborn heatmap represents pairwise correlations using a colour scale. It helps highlight stronger and weaker relationships between features and the Result target. The analysis identifies SSLfinal_State and URL_of_Anchor among the features with relatively strong positive correlation with Result.

8.4 Model-result visualizations

- Confusion matrix: displays correct and incorrect classifications by class.
- ROC curve: compares true-positive rate with false-positive rate across thresholds.
- Elbow-method plot: displays K-Means inertia for different cluster counts.
- PCA cluster scatter plot: shows four clusters in a reduced two-dimensional space.

The visualizations are generated from the notebook's data and model outputs. No separate chart values have been invented for this report. The plots support interpretation but should be considered together with the numeric metrics.

9. MODEL EVALUATION AND PERFORMANCE METRICS

The task is a binary classification problem: the target identifies a record as phishing (-1) or legitimate (1). The notebook splits the cleaned data into training and testing sets using an 80:20 ratio and stratification. It evaluates predictions with Accuracy, Precision, Recall, F1-score, ROC-AUC, a confusion matrix, and an ROC curve.

9.1 Meaning of evaluation metrics

Accuracy: proportion of all test predictions that are correct.
Precision: among records predicted as the positive class, the proportion that are actually positive.
Recall: among actual positive-class records, the proportion identified by the model.
F1-score: harmonic mean of Precision and Recall.
ROC-AUC: measures how well predicted scores separate the two classes across thresholds.
Confusion matrix: summarizes correct and incorrect predictions for each class.

9.2 Baseline Logistic Regression results

Metric	Notebook result
Accuracy	91.28%
Precision	90.00%
Recall	92.23%
F1-score	91.10%
ROC-AUC	97.38%

The baseline Logistic Regression model achieved 91.28% accuracy on the test set. Its recall was 92.23% and its F1-score was 91.10%. The ROC-AUC of 97.38% indicates strong separation in the test data. These values provide a baseline against which the later supervised models are compared.

9.3 Confusion matrix

The notebook's baseline confusion-matrix observation reports 546 correctly classified phishing records and 522 correctly classified legitimate records. It also reports 58 phishing records and 44 legitimate records misclassified. The confusion matrix helps reveal the types of errors, which is especially important because a phishing URL incorrectly classified as legitimate can create security risk.

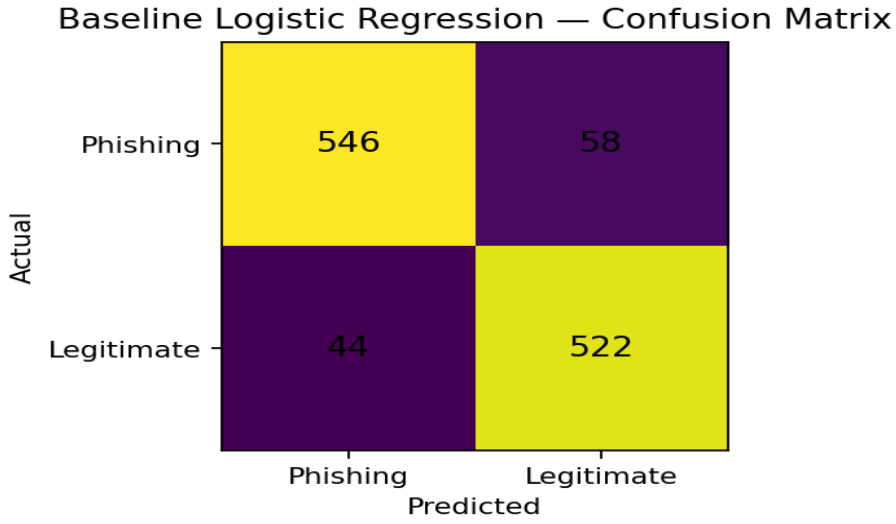


Figure 3. Baseline confusion matrix; class order: Phishing, Legitimate.

10. MODEL BUILDING

The notebook implements supervised machine learning algorithms for phishing URL classification. The cleaned input features are used to train models on the training subset; their predictions are then evaluated on the held-out test subset. Three supervised models compared in the notebook are Logistic Regression, Random Forest, and Support Vector Machine (SVM).

10.1 Logistic Regression

Logistic Regression is used as the baseline classifier. The notebook creates LogisticRegression with max_iter =1000 and random_state=42, then fits it on the training data. Its results establish a baseline for comparison: 91.28% accuracy, 90.00% precision, 92.23% recall, 91.10% F1-score, and 97.38% ROC-AUC.

10.2 Random Forest

Random Forest is an ensemble classifier that combines predictions from multiple decision trees. The notebook first trains a RandomForestClassifier with 100 estimators and random_state=42. The untuned model achieved 94.19% accuracy, 93.68% precision, 94.35% recall, 94.01% F1-score, and 98.81% ROC-AUC.

10.3 Support Vector Machine (SVM)

SVM is used as another supervised classifier. The notebook configures SVC with an RBF kernel, probability=True, and random_state=42. It is trained on the same training subset and evaluated using the same metrics, allowing the results to be compared fairly.

The model comparison is based on the outputs recorded in the notebook. The purpose is not only to find a high accuracy score, but also to examine precision, recall, F1-score, and ROC-AUC so that performance is assessed from more than one perspective.

10.4 Model training and testing

The training subset contains 4,679 rows and 30 features; the test subset contains 1,170 rows and the corresponding target labels. The same split is used for the supervised models in the notebook. The models are fitted on training data, then predictions and probability scores are used for test-set evaluation.

10.5 Hyperparameter tuning

The notebook performs hyperparameter tuning on the Random Forest classifier and evaluates the tuned model. The resulting tuned Random Forest is saved as a model file and is used by the Flask application. The project files include a saved tuned Random Forest model, and the README identifies it as the model used by the web application.

10.6 Performance comparison

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	91.28%	90.00%	92.23%	91.10%	97.38%
Random Forest	94.19%	93.68%	94.35%	94.01%	98.81%
SVM	93.59%	91.68%	95.41%	93.51%	98.23%
Tuned Random Forest	94.27%	93.54%	94.70%	94.12%	99.05%

10.7 Best model selection

The tuned Random Forest is selected as the final model because it provides the best overall balance across the comparison metrics, with the highest recorded accuracy (94.27%), F1-score (94.12%), and ROC-AUC (99.05%). SVM has the highest recall (95.41%) in the comparison, while the untuned Random Forest has slightly higher precision (93.68%).

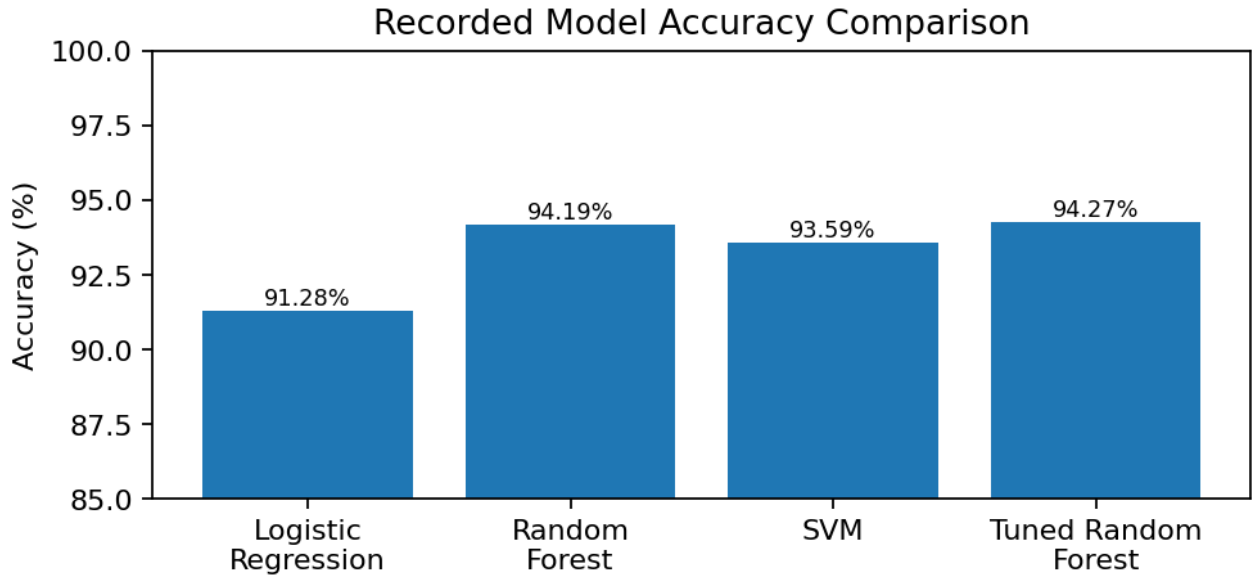


Figure 4. Accuracy comparison using recorded project metrics.

The chosen model is saved and loaded by the Flask app. The README records example predictions and confidence values; these are examples documented by the project, not a guarantee for all URLs.

11. ADVANCED TECHNIQUES AND PATTERN DISCOVERY

The project applies K-Means clustering as an unsupervised learning technique. Unlike supervised classification, K-Means groups records based on feature similarity without using the target label to form the clusters. In this project, the 30 input features are used for clustering, while Result is excluded during cluster fitting.

11.1 Feature scaling and K-Means

The notebook standardizes the input features using StandardScaler. It then applies KMeans with random_state=42 and n_init=10. The notebook tests cluster counts from K=2 to K=10 and plots inertia using the Elbow Method. Based on the notebook's chosen setting, the final clustering uses K=4.

11.2 Cluster visualization

Principal Component Analysis (PCA) reduces the feature space to two components for a scatter-plot visualization. The plot provides a simplified view of the four groups. Some overlap is visible, indicating that some records have similar feature patterns and that the clusters are not perfectly separated in the reduced two-dimensional representation.

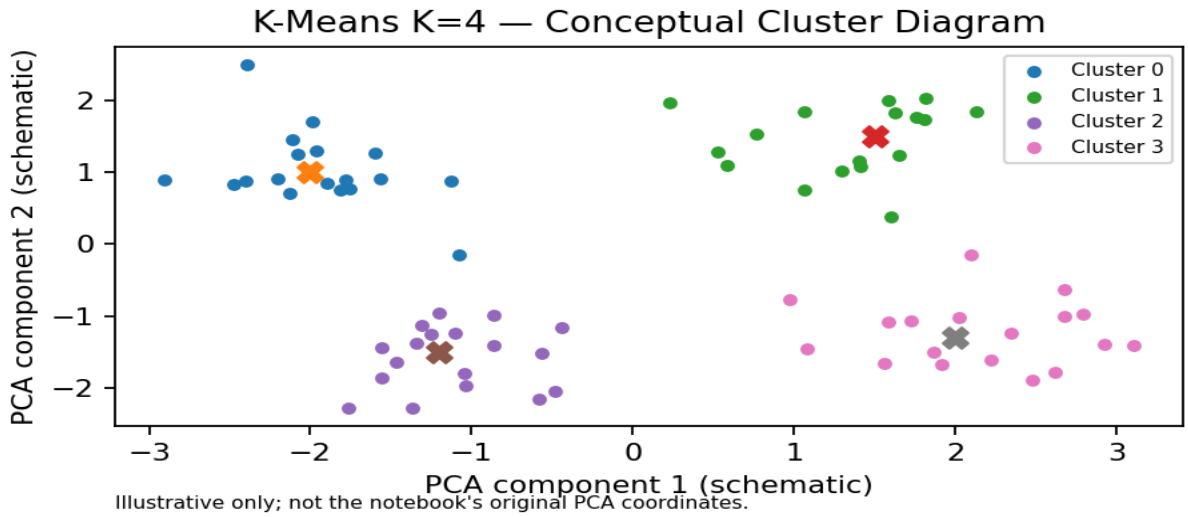


Figure 5. Conceptual K-Means illustration; not original PCA coordinates.

11.3 Cluster interpretation

Cluster	Recorded pattern in notebook
Cluster 0	Mixed distribution of phishing and legitimate records
Cluster 1	Mixed distribution of phishing and legitimate records
Cluster 2	High phishing proportion: 87.54%
Cluster 3	High legitimate proportion: 85.15%

After clustering, the notebook compares cluster membership with the known target labels to interpret the groups. This post-clustering analysis does not mean the target was used to train K-Means. The results show that some natural groups align strongly with one class, while other groups contain a mixture.
K-Means provides additional pattern discovery alongside the supervised classifiers. It is exploratory and is not the final URL prediction model used by the Flask application.

12. CONCLUSION & FUTURE WORK

12.1 Conclusion

The project “Smart AI Phishing Website & Malicious URL Blocker” demonstrates a complete machine learning workflow for phishing URL classification. The notebook loads the phishing website dataset, checks data quality, removes duplicate rows, explores class distribution and feature relationships, trains supervised classifiers, evaluates their performance, compares results, and applies K-Means clustering to discover patterns.

The original dataset contained 11,055 records. After removing 5,206 duplicate rows, 5,849 records remained with 30 input features and the Result target. No missing values or duplicate rows remained in the cleaned data. The classes were relatively balanced: 3,019 phishing records and 2,830 legitimate records.

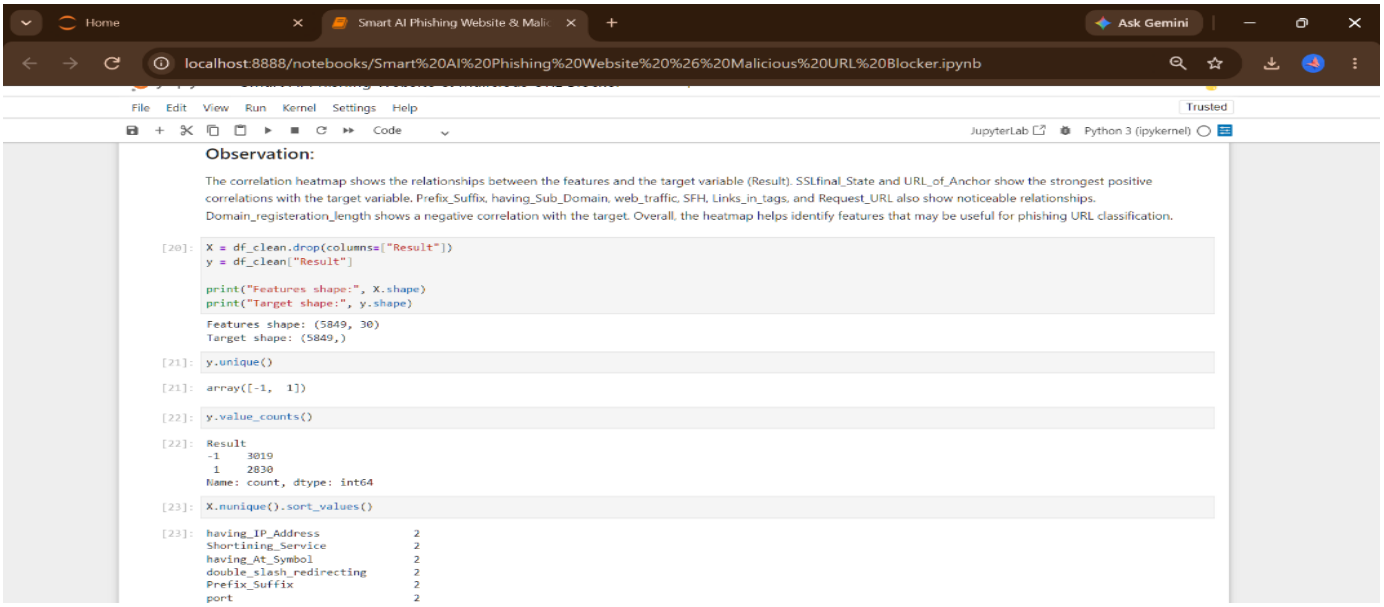
Logistic Regression was used as a baseline, and Random Forest and SVM were compared with it. The tuned Random Forest achieved the best overall balance in the recorded comparison, with 94.27% accuracy, 93.54% precision, 94.70% recall, 94.12% F1-score, and 99.05% ROC-AUC. K-Means clustering with four clusters identified groups with different phishing/legitimate distributions.

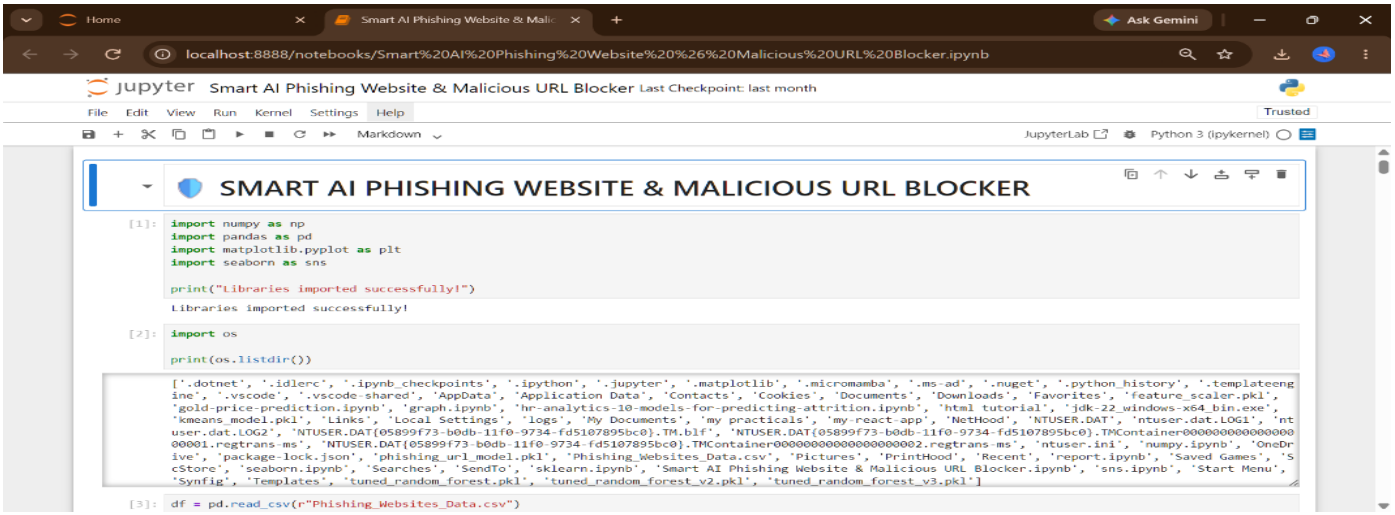
The selected tuned Random Forest model is integrated into a Flask web interface. The app accepts a URL and displays a phishing/legitimate prediction with a confidence value. This project illustrates how machine learning can assist in identifying suspicious URLs, while emphasizing that a prediction should not replace careful security practices.

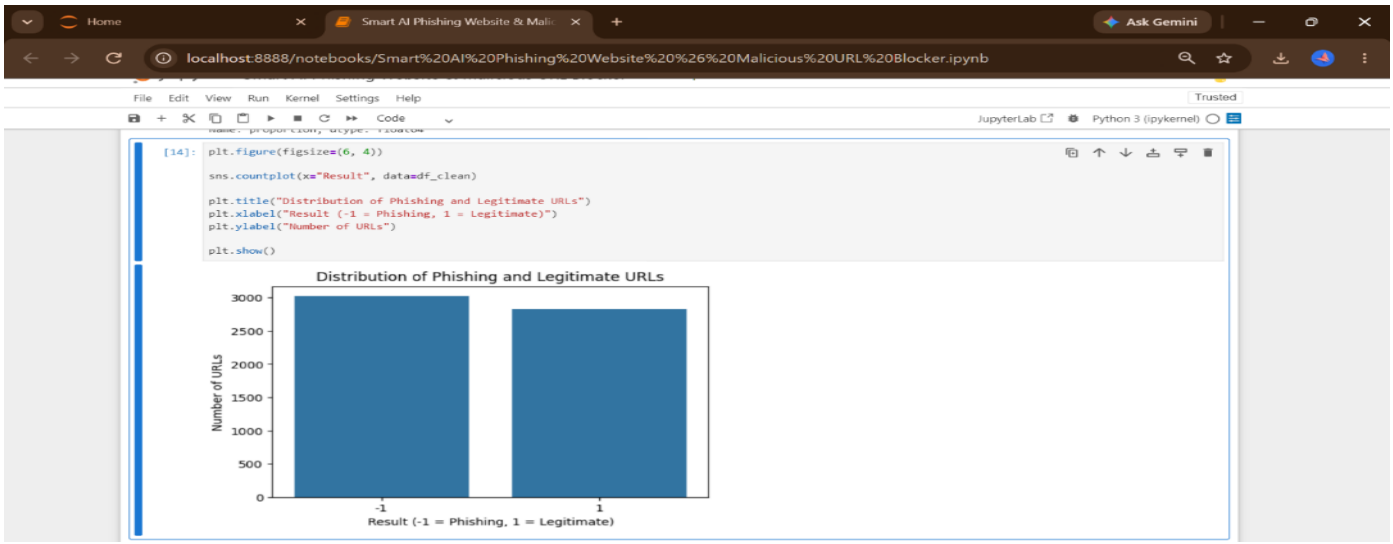
12.2 Future work

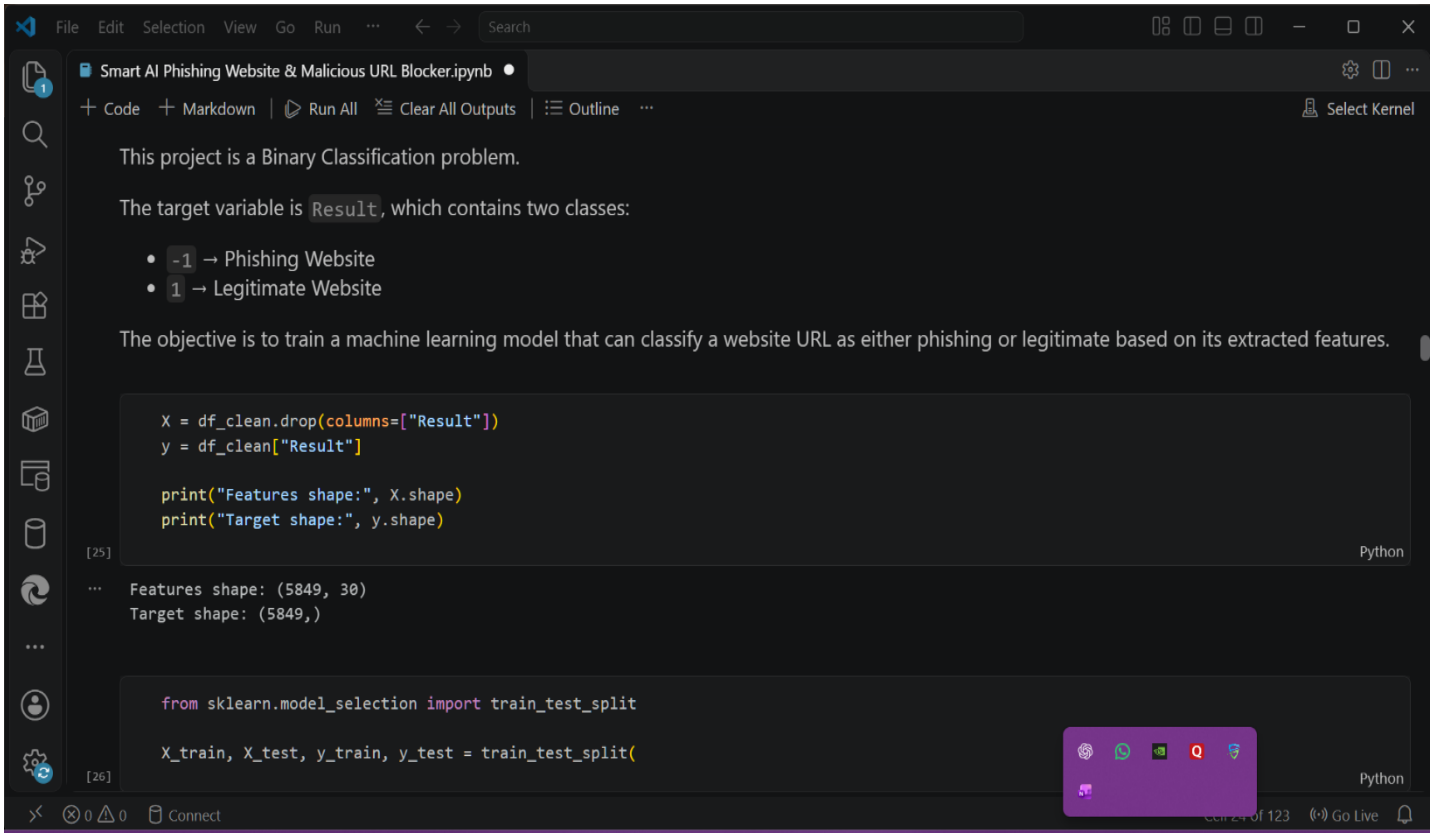
- Test the system on a larger and more recent collection of URLs to evaluate performance on changing phishing techniques.
- Improve the live URL feature-extraction pipeline and handle unavailable or slow websites robustly.
- Monitor false negatives and false positives, especially URLs incorrectly predicted as legitimate.
- Retrain and validate the model periodically when reliable new labelled data becomes available.
- Improve the user interface with clear safety guidance while explaining that model predictions are probabilistic.
- These are possible future improvements, not features claimed to be already implemented.

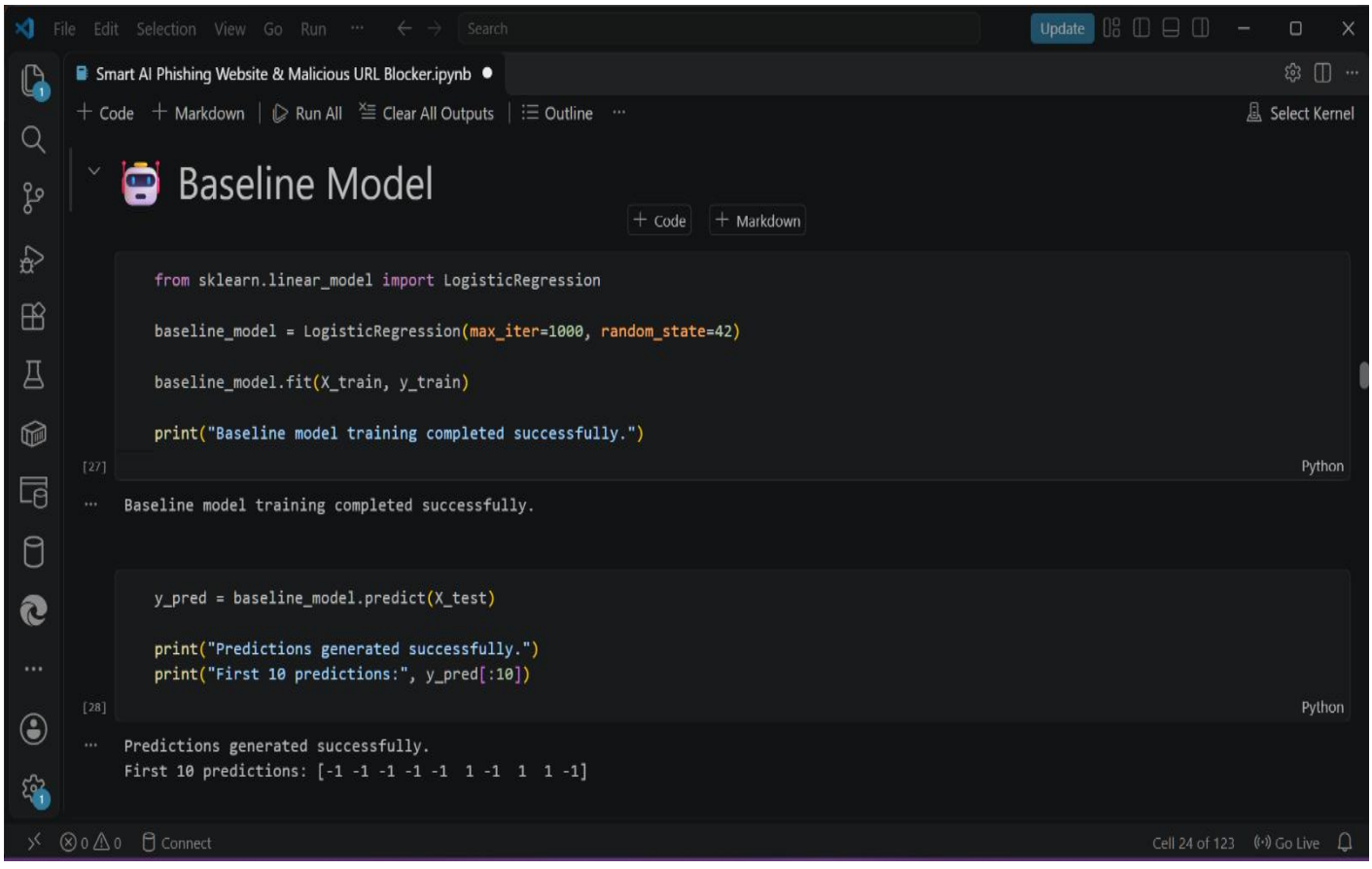
13. Application Screenshots

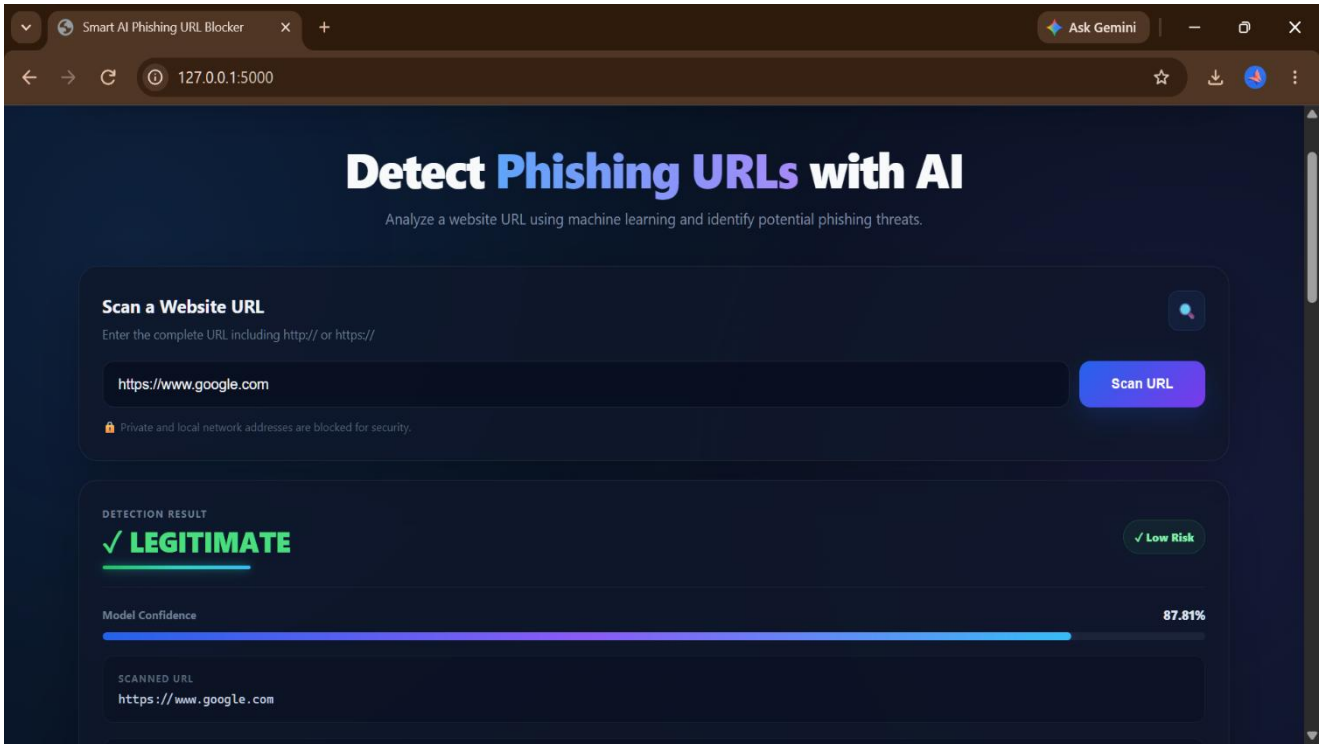


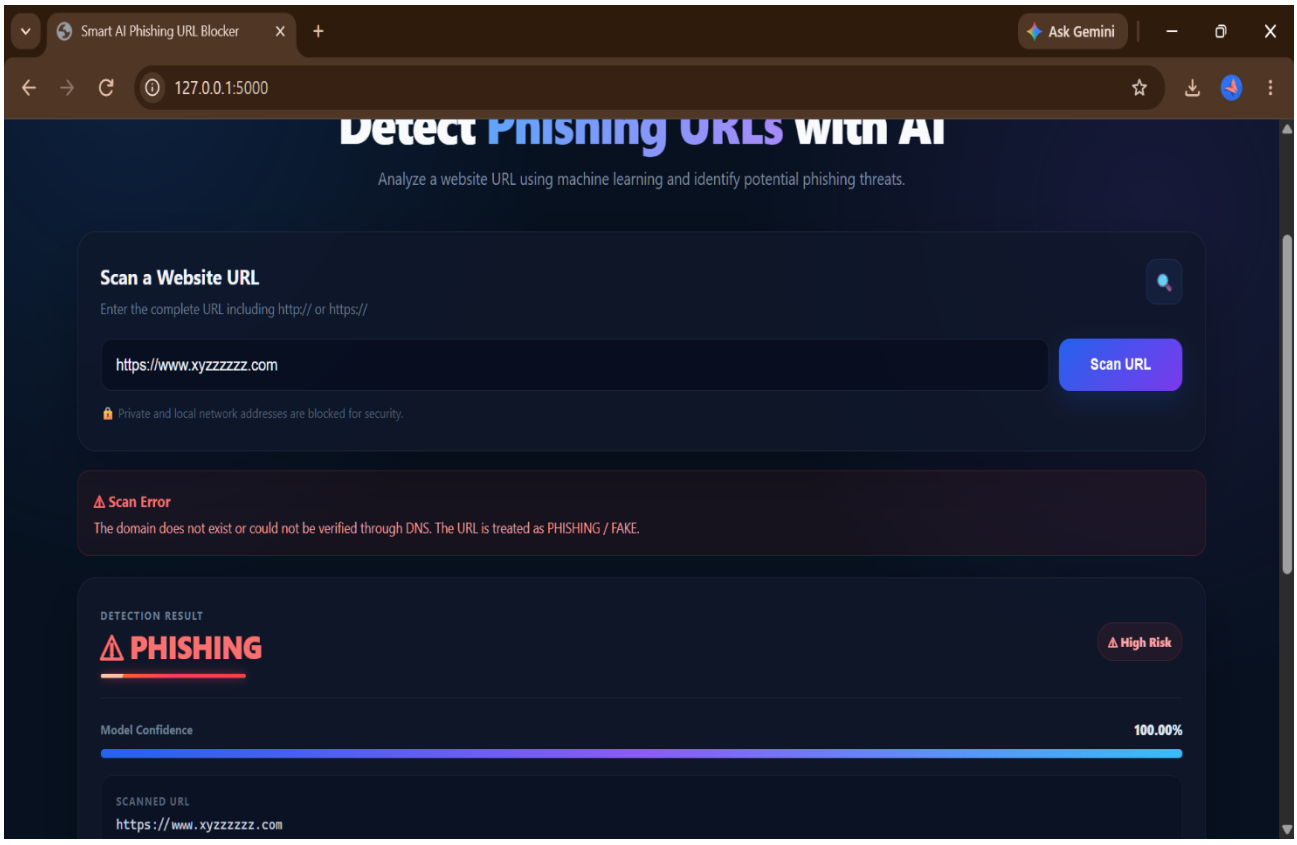





File Edit Selection View Go Run
Search
Smart AI Phishing Website & Malicious URL Blocker.ipynb
Code Markdown Run All Clear All Outputs Outline
Select Kernel
This project is a Binary Classification problem.
The target variable is Result, which contains two classes:
-1 → Phishing Website
1 → Legitimate Website
The objective is to train a machine learning model that can classify a website URL as either phishing or legitimate based on its extracted features.
X = df_clean.drop(columns=["Result"])
y = df_clean["Result"]
print("Features shape:", X.shape)
print("Target shape:", y.shape)
[25]
Python
Features shape: (5849, 30)
Target shape: (5849,)
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
[26]
Python
Connect
Go Live


File Edit Selection View Go Run
Search
Update
Smart AI Phishing Website & Malicious URL Blocker.ipynb
Code Markdown Run All Clear All Outputs Outline
Select Kernel
Baseline Model
Code Markdown
from sklearn.linear_model import LogisticRegression
baseline_model = LogisticRegression(max_iter=1000, random_state=42)
baseline_model.fit(X_train, y_train)
print("Baseline model training completed successfully.")
[27]
Python
Baseline model training completed successfully.
y_pred = baseline_model.predict(X_test)
print("Predictions generated successfully.")
print("First 10 predictions:", y_pred[:10])
[28]
Python
Predictions generated successfully.
First 10 predictions: [-1 -1 -1 -1 -1 1 -1 1 1 -1]
Connect
Cell 24 of 123
Go Live


Smart AI Phishing URL Blocker
Ask Gemini
127.0.0.1:5000
Detect Phishing URLs with AI
Analyze a website URL using machine learning and identify potential phishing threats.
Scan a Website URL
Enter the complete URL including http:// or https://
https://www.google.com
Scan URL
Private and local network addresses are blocked for security.
DETECTION RESULT
✓ LEGITIMATE
✓ Low Risk
Model Confidence
87.81%
SCANNED URL
https://www.google.com


Smart AI Phishing URL Blocker
Ask Gemini
127.0.0.1:5000
Analyze a website URL using machine learning and identify potential phishing threats.
Scan a Website URL
Enter the complete URL including http:// or https://
https://www.xyzzzzzz.com
Scan URL
Private and local network addresses are blocked for security.
⚠ Scan Error
The domain does not exist or could not be verified through DNS. The URL is treated as PHISHING / FAKE.
DETECTION RESULT
⚠ PHISHING
⚠ High Risk
Model Confidence
100.00%
SCANNED URL
https://www.xyzzzzzz.com

14. BIBLIOGRAPHY

The referenced resources helped in understanding phishing website datasets, building Machine Learning models, analyzing data, and visualizing results. These resources support the development of a system for detecting phishing websites and improving online security.

- https://archive.ics.uci.edu/dataset/327/phishing+websites
- https://scikit-learn.org/stable/
- https://pandas.pydata.org/docs/
- https://matplotlib.org/stable/
- https://seaborn.pydata.org/